

Processamento de Cadeias: Compactação de Dados

Estruturas de Dados

Prof. Dr. Anderson Bessa da Costa

Universidade Federal de Mato Grosso do Sul

Introdução

- **Cadeia** é uma sequência qualquer de elementos, denominados **caracteres**
- **Caracteres** são elementos escolhidos de um conjunto denominado **alfabeto**
 - Por exemplo, 0110010 é uma **cadeia** com **alfabeto** {0, 1}

Introdução (Cont.)

- **Cadeias** aparecem, em computação, no processamento de textos, palavras, mensagens, códigos etc.
 - Aplicações como dicionários, mensagens de correio eletrônico, sequenciamento de DNA em biologia computacional, criptografia de textos confidenciais ...

Processamento de Cadeias

1. Casamento de cadeias

- Verificar se uma cadeia contém outra cadeia
- Dadas duas cadeias de caracteres, verificar se a primeira delas contém a segunda

2. Codificação de mensagens

- Minimizar o comprimento total
- Dada uma cadeia de caracteres, codificá-la através da atribuição de códigos a seus caracteres, de modo a minimizar o comprimento total

Forma de Armazenamento de Dados

- O que caracteriza o **processamento de cadeias** não é o tipo de problema tratado, mas sim a forma de **armazenamento dos dados**
 - Por exemplo, o problema do casamento de cadeias pode ser resolvido através de métodos como busca digital
- Supõe-se que os caracteres estejam dispostos **sequencialmente em uma lista**, sem outra forma de estrutura

Codificação de Mensagens (Compactação de Dados)

Compactação de Dados

- Suponha que se deseje armazenar um arquivo **grande** em algum tipo de memória
- Deseja-se **minimizar** o espaço de memória utilizado
 - **Codificar** o conteúdo do arquivo de maneira apropriada
 - Se o arquivo **codificado** for menor do que o original, pode-se armazenar a versão **codificada**
 - Ganho de memória

Tabela de Códigos

- Naturalmente, uma **tabela de códigos** seria também armazenada, para permitir a decodificação do arquivo
 - Utilizada pelos **algoritmos de codificação e decodificação**
- Este problema é conhecido como **compactação de dados**

Compactar Arquivo

- Seja o arquivo a ser compactado consiste em um texto alfabético:
 - BBBEAAAFFHHHHCBMMALLCDDBBBBBBBBBCC
- Para compactar esse texto, pode-se empregar uma ideia bastante simples
 - Qual??

Algoritmo de Frequência de Caracteres

- BBBEAAAFFHHHHCBMMALLCDDBBBBBBBBBCC
- Determina-se a quantidade de símbolos idênticos consecutivos existentes no texto
- Cada uma das subsequências máximas de símbolos idênticos do texto é substituída por um número indicando a frequência do símbolo em questão
 - 3B1E4A2F5H1C1B2M1A3L1C2D7B2C

Algoritmo de Frequência de Caracteres (2)

- BBBEAAAAFFHHHHHCBMMALLCDDBBBBBBBBBCC
- 3B1E4A2F5H1C1B2M1A3L1C2D7B2C
- Pode-se adotar como regra de compactação que a ausência do número representativo da frequência de um símbolo significa frequência igual a um:
 - 3BE4A2F5HCB2MA3LC2D7B2C

Algoritmo de Frequência de Caracteres (3)

- BBBEAAAAFFHHHHHCBMMALLLCDDBBBBBBBCC
- 3B1E4A2F5H1C1B2M1A3L1C2D7B2C
- 3BE4A2F5HCB2MA3LC2D7B2C
- Pode-se empregar algum símbolo especial, por exemplo @, para anunciar que o número imediatamente após representa um símbolo e não uma frequência:
 - AAA33333BA6666888DDDDDDDD999999999999AABBB
 - 3A5@3BA4@63@87D11@92A3B

O Algoritmo de Huffman

Problema de Compactação Ótima

- Veremos um método para codificar um texto com compactação **ótima** dentro de certos critérios
- Supõe-se que o texto seja constituído de um conjunto de **símbolos** (ou caracteres) $S = \{s_1, \dots, s_n\}$, $n > 1$
- É conhecida a **frequência** f_i de cada **símbolo** s_i no texto
- Deseja-se atribuir um código a cada **símbolo**, de modo a compactar o texto todo
- A restrição que se coloca é que nenhum código seja prefixo de algum outro
 - O que isso nos lembra??

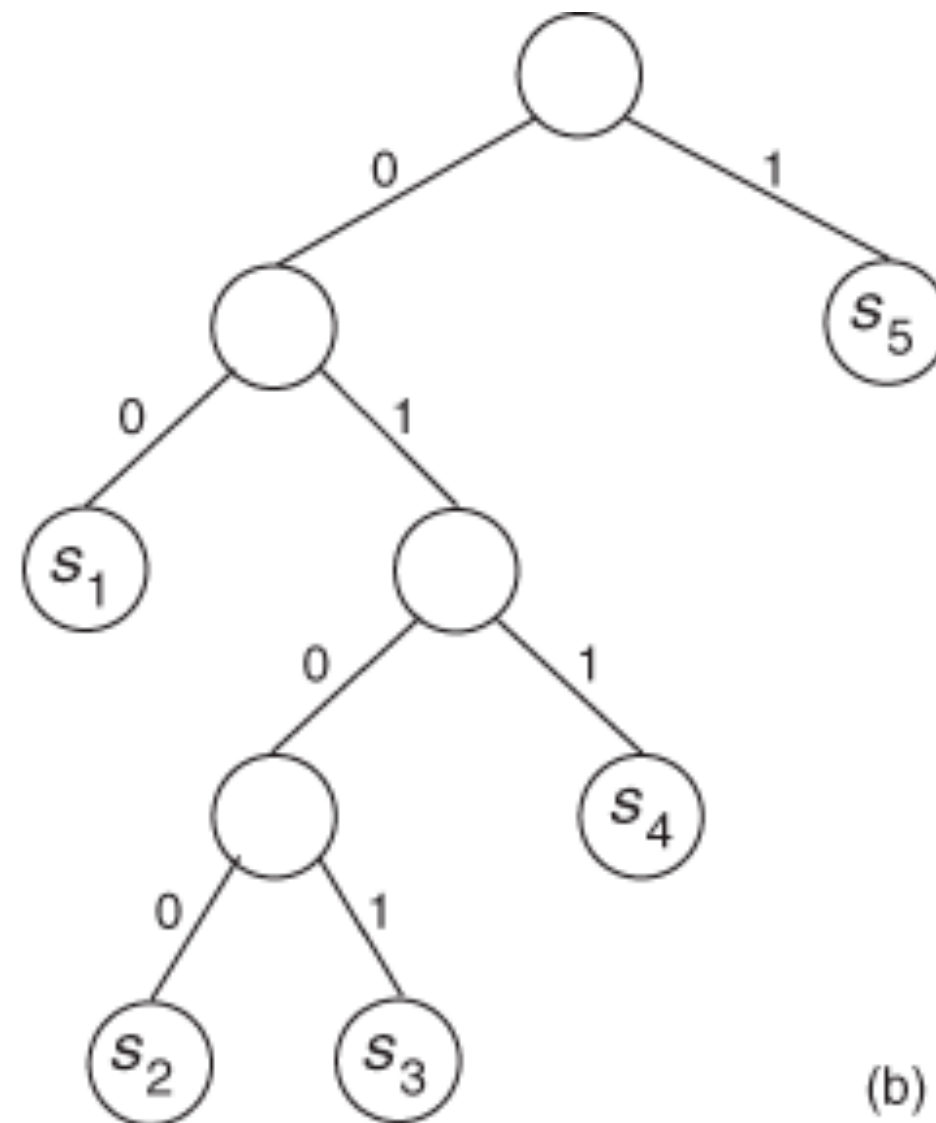
Árvore Binária de Prefixo

- Ou seja, os códigos procurados são aqueles dados por uma **árvore binária de prefixo**
- Há uma correspondência entre o conjunto das **chaves** e o das **folhas** da árvore
 - Cada **chave** é unicamente representada por uma **folha**
 - Codificação binária dessa chave corresponde ao caminho da raiz até essa folha

Figura: Árvore de Prefixo

<i>símbolo</i>	<i>código</i>
s_1	00
s_2	0100
s_3	0101
s_4	011
s_5	1

(a)



(b)

Codificar

- Suponha o seguinte texto:
 - $s_4 s_3 s_3 s_1 s_3 s_1 s_4 s_5 s_1 s_3 s_3 s_3 s_3 s_3 s_2 s_3 s_5 s_2 s_2 s_2 s_4$
- Utilizando-se da tabela dada, obtém-se a seguinte **codificação**:
 - 011010101010001010001110001010101010101010101010
10001011010001000100011

Decodificar

- Para **decodificar**, percorre-se da esquerda para a direita, ao mesmo tempo em que a árvore é percorrida, repetidamente, da raiz para as folhas
 - Toda vez que uma folha é atingida, um símbolo **decodificado** foi encontrado
 - Reinicia-se o percurso na árvore a partir da raiz
- Uma vez conhecida a **árvore de prefixo** correspondente, o processo de **codificação** ou **decodificação** pode ser realizado em um número de passos linear no tamanho da sequência binária codificada

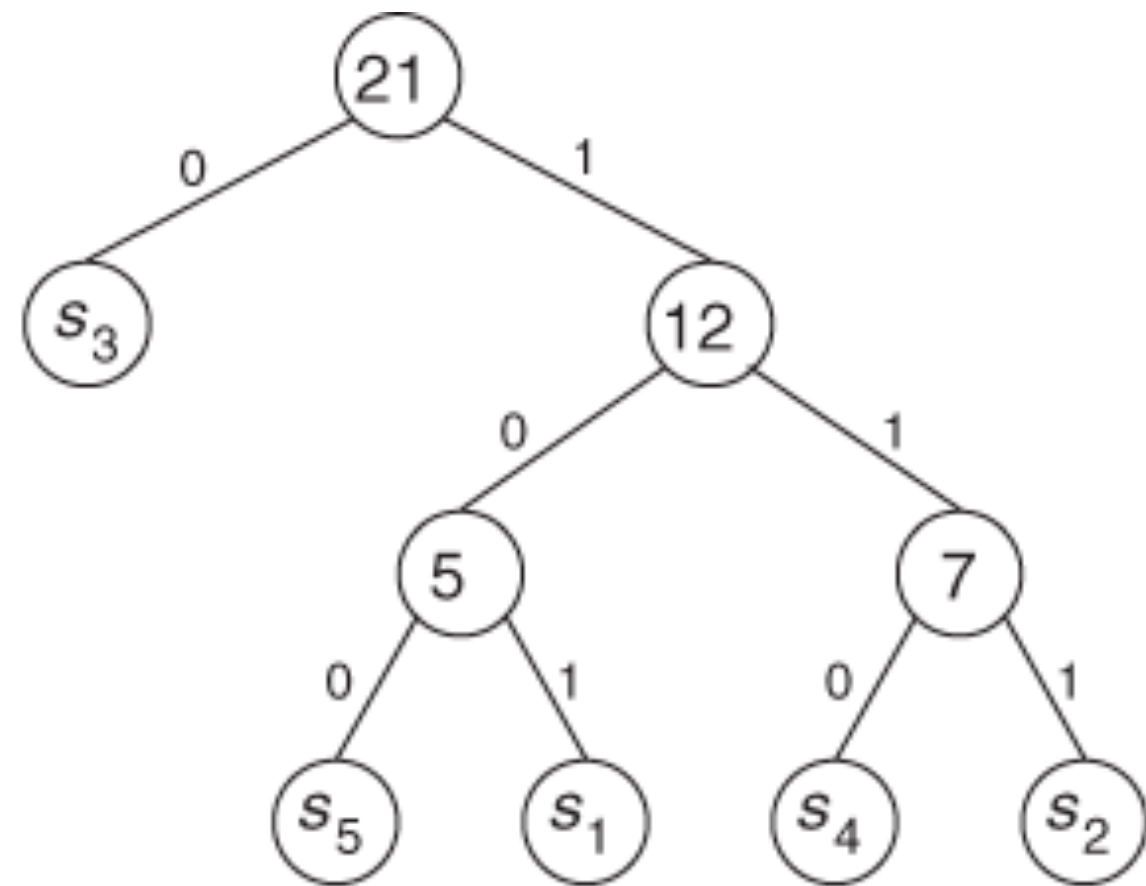
Comprimento do Texto Codificado

- O **comprimento** da sequência binária produzida por uma árvore binária de prefixo T é denominado **custo** de T e denotado por $c(T)$
 - No exemplo anterior, $c(T) = 69$
- O objetivo consiste, então, em minimizar $c(T)$
 - Ou seja, construir T de modo que $c(T)$ seja mínimo
- Uma árvore de prefixo que satisfaça esta condição é denominada mínima ou **árvore de Huffman**

Figura: Árvore de Huffman

<i>símbolo</i>	<i>frequência</i>	<i>código</i>
s_1	3	101
s_2	4	111
s_3	9	1
s_4	3	110
s_5	2	100

(a)



(b)

Custo Mínimo

- **Codificando-se** o texto do exemplo com a árvore mínima, obtém-se a sequência:
 - 1100010101011101001010000011101001111111111110
- Nesse caso, $c(T) = 45$
 - Melhoria considerável

Árvore Mínima

- Seja T uma árvore de prefixo correspondente a um dado texto
- Cada símbolo s_i ocorre f_i vezes
- Seja ℓ_i o comprimento do código binário do símbolo s_i
- Cada símbolo s_i contribui com $f_i \ell_i$ unidades no custo $c(T)$

$$\text{Logo, } c(T) = \sum_{1 \leq i \leq n} f_i \ell_i$$

- Este valor corresponde ao conhecido **comprimento de caminho externo ponderado** de T
- Se todas as frequências são idênticas, então T é uma árvore binária completa e $c(T)$ é o **comprimento do caminho externo** de T

Construir Árvore de Huffman

- Dado conjunto de $n > 1$ símbolos s_i de frequências f_i , $1 \leq i \leq n$
- **Algoritmo ganancioso** (*greedy*, guloso)
- Construção da árvore mínima de forma iterativa
- Seja T' uma subárvore binária de prefixo. Isto é, as folhas de T' são símbolos selecionados dentre $\{s_1, \dots, s_n\}$
- Define-se $f(T')$, a **frequência** de T'
 - Soma das frequências dos símbolos s_i presentes nas folhas de T'

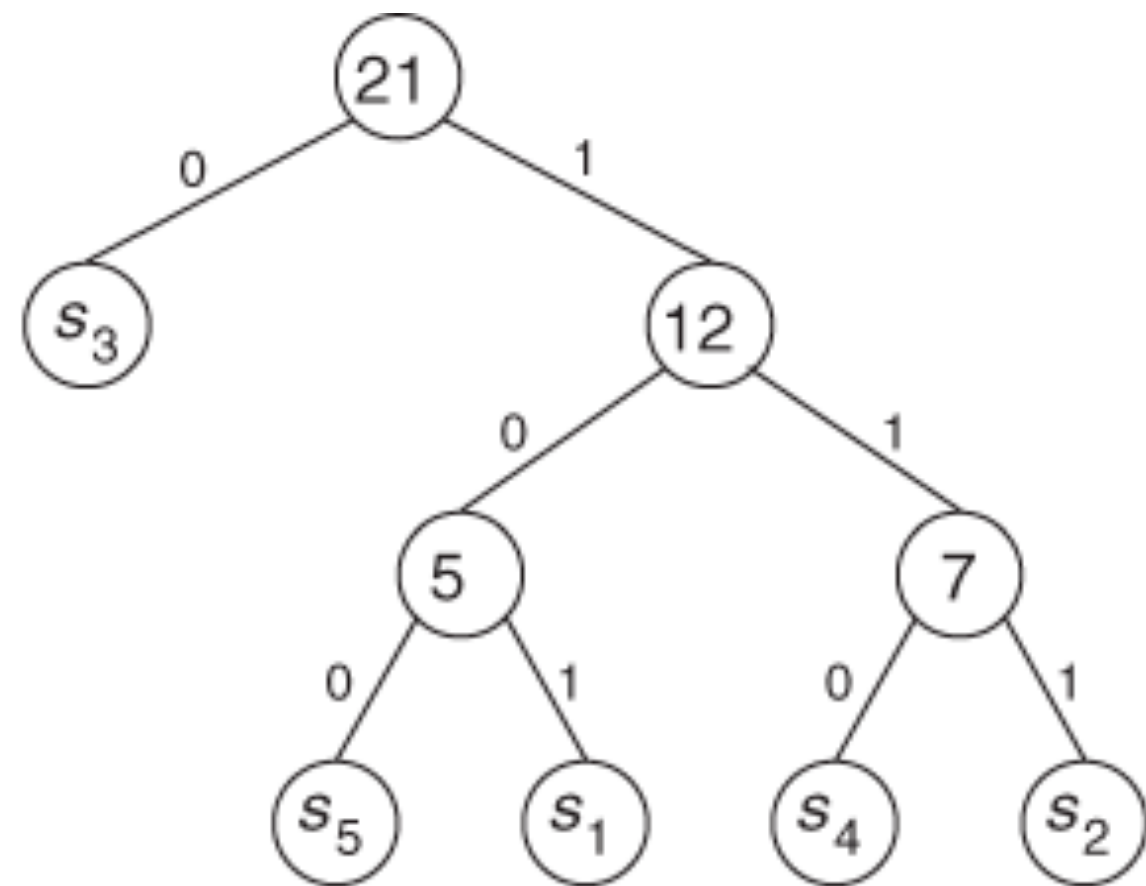
Figura: Nós internos

- Os números representados nos nós internos da árvore correspondem às frequências das subárvores de raiz nos respectivos nós

símbolo *frequência* *código*

s_1	3	101
s_2	4	111
s_3	9	1
s_4	3	110
s_5	2	100

(a)



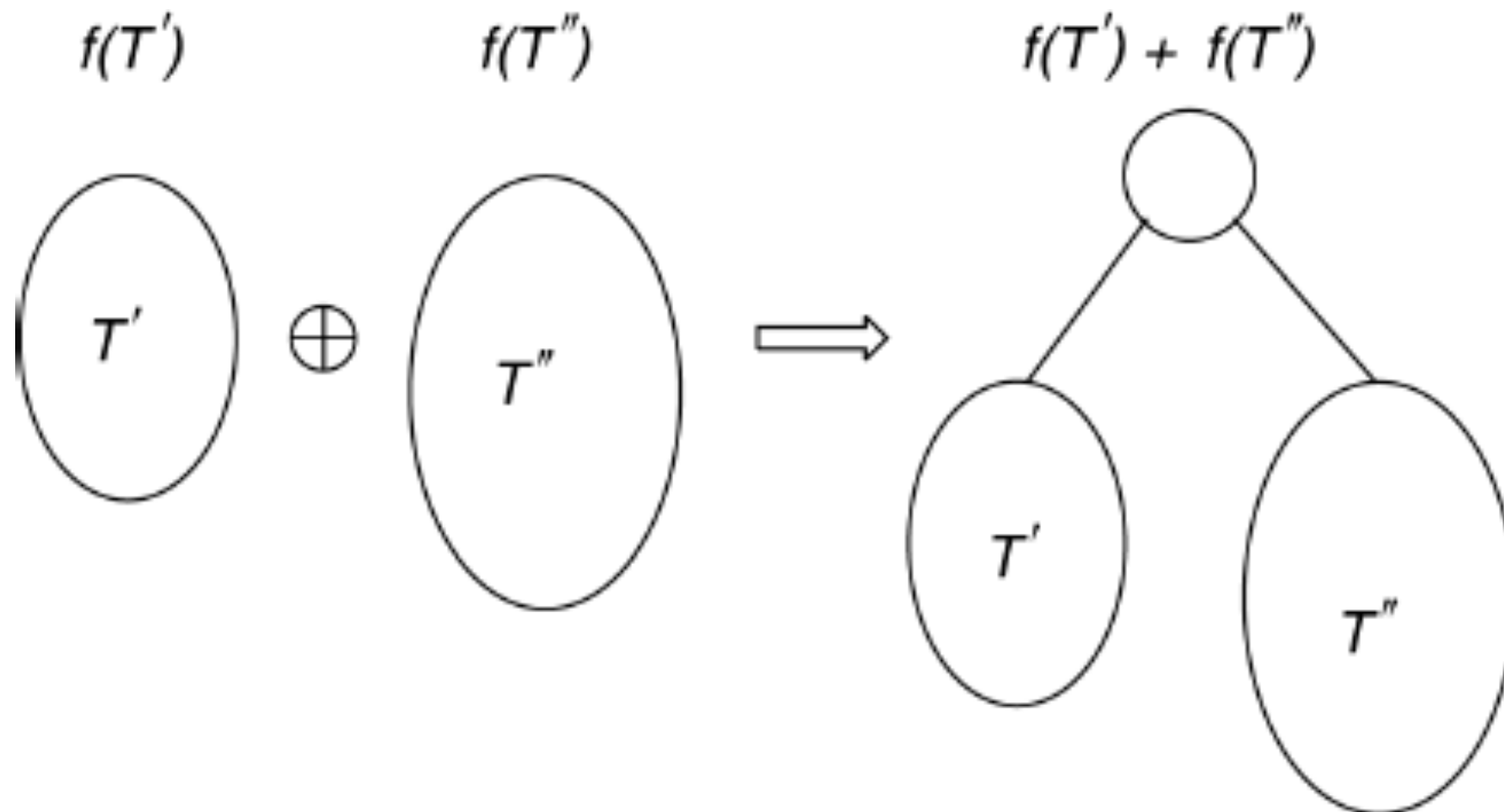
(b)

Processo de Construção da Árvore de Huffman

- Definem-se n subárvores, cada qual consistindo em um único nó contendo o símbolo s_i , $1 \leq i \leq n$
- A frequência de cada uma dessas n subárvores é, pois, igual à frequência do símbolo a ela correspondente
- Iterativamente, seleciona as duas subárvores T' e T'' , tais que $f(T')$ e $f(T'')$ sejam as duas menores frequências
- Funde-se essas duas subárvores de custo mínimo (operação $\oplus$)
- O algoritmo termina quando restar apenas uma única subárvore
 - $n - 1$ execuções da operação $\oplus$

Figura: Operação $\oplus$

- A frequência $f(T' \oplus T'')$ da nova subárvore $T' \oplus T''$ é igual a $f(T') + f(T'')$



Algoritmo: Construção da Árvore de Huffman

para $i := 1, n - 1$ **faça**

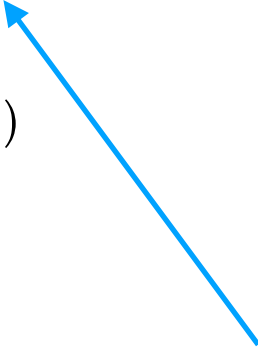
minimo(T' , f , F)

minimo(T'' , f , F)

$T := T' \oplus T''$

$f(T) := f(T') + f(T'')$

inserir(T , f , F)



Deve-se utilizar, portanto, uma estrutura de dados que suporte as operações básicas de minimização, inclusão e exclusão.

Complexidade

- Cada operação de minimização ou inclusão na lista de prioridade pode ser efetuada em $O(\log n)$ passos
- A operação $\oplus$ requer apenas um número constante de passos
- Há um total de $n - 1$ iterações
- Logo, a complexidade é $O(n \log n)$ passos

Aplicação do Algoritmo de Huffman

símbolo *frequência*

s_1 3

s_2 4

s_3 9

s_4 3

s_5 2

inicialização

s_1

s_2

s_3

s_4

s_5

- Quadro ...

Árvore de Custo Mínimo

- Lema 12.1 (pág. 290): Sejam s_i símbolos com frequências f_i , $1 \leq i \leq n$, $n > 1$, tais que f_1 e f_2 são as duas menores frequências. Então existe uma árvore de Huffman para esses símbolos, em que os nós correspondentes a s_1 e s_2 são irmãos localizados no último nível da árvore.

Corretude do Algoritmo

- Lema 12.2 (pág. 291): Seja T a árvore construída pelo algoritmo de Huffman para as frequências $f_1, \dots, f_n$, $n > 1$. Então, T é mínima

Considerações Finais

- O método de **Huffman** é uma ferramenta importante para a compactação de dados
- A restrição da utilização de códigos de prefixo é perfeitamente razoável
- E, para os códigos de prefixo, a árvore de **Huffman** é comprovadamente o **melhor** método
- **Desvantagem** é que um possível erro de transmissão em apenas um dos dígitos no texto binário codificado pode ocasionar sérios problemas no processo de decodificação

Referências

- CORMEN, T. H. [et al]. Algoritmos: teoria e prática. 3a ed. Rio de Janeiro: Elsevier, 2012
- SZWARCFITER, Jayme Luiz; MARKENZON, Lilian. Estruturas de Dados e seus Algoritmos. Edição: 3a. Editora: LTC. 2010